DESARROLLO DE SOFTWARE PARA MOVILES DSM104

PRIMER PROYECTO KOTLIN:

Proyecto carrito de compras en Kotlin con facturación en modo consola

ENLACE A REPOSITORIO EN GITHUB:

AQUÍ

Frank Alberto Hernández Silva	HS171707
Andrea Marcela Rico Figueroa	RF160050
Marcos Daniel Ibáñez Guevara	IG243224

San Salvador, 29 de marzo de 2026

PROYECTO CARRITO DE COMPRAS EN KOTLIN CON FACTURACIÓN EN MODO CONSOLA

1- Carrito de compras en consola:

```
"C:\Program Files\Java\jdk-14.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2022.1.4\lib\idea_rt.jar=63263:C:\Program Files\JetBrains\IntelliJ IDEA 2022.
BIENVENIDO AL SISTEMA DE VENTAS KOTLIN

--- MENÚ DE OPCIONES ---
1. Ver Inventario
2. Agregar al Carrito
3. Ver Carrito
4. Quitar del Carrito
5. Finalizar Compra e Imprimir Factura
6. Salir
Seleccione:
```

2- Inventario:

```
--- MENÚ DE OPCIONES ---
1. Ver Inventario
2. Agregar al Carrito
3. Ver Carrito
4. Quitar del Carrito
5. Finalizar Compra e Imprimir Factura
6. Salir
Seleccione: 1

CATÁLOGO DISPONIBLE:
ID: 1 | Laptop | Precio: $800.0 | Stock: 5
ID: 2 | Mouse | Precio: $25.0 | Stock: 20
ID: 3 | Teclado | Precio: $45.0 | Stock: 15
ID: 4 | Monitor | Precio: $150.0 | Stock: 10
```

3- Agregar al carrito:

```
--- MENÚ DE OPCIONES ---
1. Ver Inventario
2. Agregar al Carrito
3. Ver Carrito
4. Quitar del Carrito
5. Finalizar Compra e Imprimir Factura
6. Salir
Seleccione: 2
Ingrese ID del producto: 1
Ingrese Cantidad: 3
¡Laptop agregado correctamente!
```

4- Ver carrito:

```
--- MENÚ DE OPCIONES ---
1. Ver Inventario
2. Agregar al Carrito
3. Ver Carrito
4. Quitar del Carrito
5. Finalizar Compra e Imprimir Factura
6. Salir
Seleccione: 3

--- CONTENIDO DEL CARRITO ---
Producto        Cant.      Precio U.       Total
------------------------------------------------------------
Laptop          3          $800.00         $2400.00
Teclado         5          $45.00          $225.00
Mouse           4          $25.00          $100.00
------------------------------------------------------------
SUBTOTAL DEL CARRITO: $2725.00
```

5- Quitar del carrito:

```
--- MENÚ DE OPCIONES ---
1. Ver Inventario
2. Agregar al Carrito
3. Ver Carrito
4. Quitar del Carrito
5. Finalizar Compra e Imprimir Factura
6. Salir
Seleccione: 4
Ingrese ID del producto a eliminar: 1
Producto removido.
```

6- Finalizar compra e imprimir facturación:

```
=========================================
           FACTURA FINAL
=========================================

--- CONTENIDO DEL CARRITO ---
Producto        Cant.      Precio U.       Total
-----------------------------------------------------------
Teclado         5          $45.00          $225.00
Mouse           4          $25.00          $100.00
-----------------------------------------------------------
SUBTOTAL DEL CARRITO: $325.00

IVA (13%):          $42.25
TOTAL A PAGAR:      $367.25
=========================================
Factura generada y enviada con éxito por correo (Simulado).

¿Desea realizar otra compra diferente? (S/N):
```

CÓDIGO

```
import java.io.File
import java.time.LocalDateTime
import java.util.Scanner

// 1. CLASE PRODUCTO (POO)
data class Producto(
    val id: Int,
    val nombre: String,
    var precio: Double,
    var stock: Int
)

// 2. CLASE CARRITO (Manejo de Colecciones y Cálculos por Producto)
class Carrito {
    private var items = mutableListOf<Pair<Producto, Int>>()

    fun agregar(producto: Producto, cantidad: Int): String {
        return if (producto.stock >= cantidad) {
            items.add(producto to cantidad)
            producto.stock -= cantidad
            "¡${producto.nombre} agregado correctamente!"
        } else {
            val error = "Error: Solo quedan ${producto.stock} unidades de
${producto.nombre}"
            Logger.registrarError(error)
            error
        }
    }

    fun eliminar(idProducto: Int): Boolean {
        val item = items.find { it.first.id == idProducto }
        return if (item != null) {
            item.first.stock += item.second
            items.remove(item)
            true
        } else {
            Logger.registrarError("Intento de eliminar producto ID $idProducto inexistente.")
            false
        }
    }

    fun obtenerItems() = items

    fun calcularSubtotal() = items.sumOf { it.first.precio * it.second }

    // Función para limpiar el carrito después de una compra
```

UNIVERSIDAD DON BOSCO

```
    fun vaciar() {
        items = mutableListOf()
    }
}

// 3. REQUERIMIENTOS TÉCNICOS (Archivo Log y Facturación)
object Logger {
    fun registrarError(mensaje: String) {
        try {
            val file = File("errores.txt")
            file.appendText("[${LocalDateTime.now()}] ERROR: $mensaje\n")
        } catch (e: Exception) {
            println("No se pudo escribir en el log.")
        }
    }
}

// 4. VISUALIZACIÓN DETALLADA DEL CARRITO
fun mostrarCarritoDetallado(carrito: Carrito) {
    val items = carrito.obtenerItems()
    if (items.isEmpty()) {
        println("\nEl carrito está vacío.")
        return
    }

    println("\n--- CONTENIDO DEL CARRITO ---")
    println(String.format("%-15s %-10s %-15s %-10s", "Producto", "Cant.", "Precio U.",
"Total"))
    println("-----------------------------------------------------------")

    items.forEach { (prod, cant) ->
        val totalPorProducto = prod.precio * cant
        println(String.format("%-15s %-10d $%-14.2f $%-10.2f",
            prod.nombre, cant, prod.precio, totalPorProducto))
    }
    println("-----------------------------------------------------------")
    println("SUBTOTAL DEL CARRITO: $${String.format("%.2f",
carrito.calcularSubtotal())}\n")
}

fun imprimirFacturaFinal(carrito: Carrito) {
    val subtotal = carrito.calcularSubtotal()
    val iva = subtotal * 0.13
    val total = subtotal + iva

    println("\n========================================")
    println("          FACTURA FINAL                 ")
    println("========================================")
```

```
    mostrarCarritoDetallado(carrito)
    println("IVA (13%):          $${String.format("%.2f", iva)}")
    println("TOTAL A PAGAR:      $${String.format("%.2f", total)}")
    println("========================================")
    println("Factura generada y enviada con éxito por correo (Simulado).")
}

// 5. INTERFAZ DE USUARIO (Menú Principal)
fun main() {
    val sc = Scanner(System.`in`)
    val inventario = mutableListOf(
        Producto(1, "Laptop", 800.0, 5),
        Producto(2, "Mouse", 25.0, 20),
        Producto(3, "Teclado", 45.0, 15),
        Producto(4, "Monitor", 150.0, 10)
    )
    val carrito = Carrito()
    var appCorriendo = true

    println("BIENVENIDO AL SISTEMA DE VENTAS KOTLIN")

    while (appCorriendo) {
        println("\n--- MENÚ DE OPCIONES ---")
        println("1. Ver Inventario")
        println("2. Agregar al Carrito")
        println("3. Ver Carrito")
        println("4. Quitar del Carrito")
        println("5. Finalizar Compra e Imprimir Factura")
        println("6. Salir")
        print("Seleccione: ")

        val input = sc.next()
        val opcion = input.toIntOrNull()

        if (opcion == null) {
            println("!!! Error: '$input' no es un número válido.")
            Logger.registrarError("Entrada no válida: $input")
            continue
        }

        when (opcion) {
            1 -> {
                println("\nCATÁLOGO DISPONIBLE:")
                inventario.forEach { println("ID: ${it.id} | ${it.nombre} | Precio: $${it.precio} |
Stock: ${it.stock}") }
            }
            2 -> {
                print("Ingrese ID del producto: ")
```

```
            val idInput = sc.next().toIntOrNull()
            print("Ingrese Cantidad: ")
            val cantInput = sc.next().toIntOrNull()

            if (idInput != null && cantInput != null) {
                val p = inventario.find { it.id == idInput }
                if (p != null) println(carrito.agregar(p, cantInput))
                else println("Producto no encontrado.")
            } else {
                println("Error en los datos ingresados.")
                Logger.registrarError("Error al ingresar ID o cantidad.")
            }
        }
        3 -> mostrarCarritoDetallado(carrito)
        4 -> {
            print("Ingrese ID del producto a eliminar: ")
            val idRemover = sc.next().toIntOrNull()
            if (idRemover != null && carrito.eliminar(idRemover)) println("Producto
removido.")
            else println("No se pudo remover el producto.")
        }
        5 -> {
            if (carrito.obtenerItems().isNotEmpty()) {
                imprimirFacturaFinal(carrito)

                // REQUISITO: Preguntar si desea continuar
                print("\n¿Desea realizar otra compra diferente? (S/N): ")
                val respuesta = sc.next()
                if (respuesta.lowercase() == "s") {
                    carrito.vaciar() // Limpiamos el carrito para una nueva compra
                    println("\nCarrito vaciado. Puede comenzar una nueva compra.")
                } else {
                    appCorriendo = false
                }
            } else {
                println("El carrito está vacío. Agregue productos antes de pagar.")
            }
        }
        6 -> appCorriendo = false
        else -> println("Opción fuera de rango.")
    }
  }
  println("Gracias por usar nuestro sistema. ¡Vuelva pronto!")
}
```